

Anton Bobrov

Senior Backend / Platform Engineer

Location: London

E-mail: baverman@gmail.com

GitHub: <https://github.com/baverman>

Telegram: [@bvrnm](https://t.me/bvrnm)

Profile

I am a senior backend/platform engineer with 15+ years of experience, using Python as my main language. I build HTTP APIs, backend services, and infrastructure tooling for production systems where performance, reliability, and maintainability matter.

I also work comfortably outside Python when needed. I have experience with Java, C++, and C, and strong systems programming knowledge: binary formats, network programming, protocols, process management, and integrating fast C/C++ code with Python.

I prefer simple implementations, practical observability, and focused test suites that provide fast feedback during code changes. I have deep experience investigating and fixing performance issues in production systems.

I have experience with model checking (TLA+, SPIN) in the context of distributed systems.

I can work effectively with LLM-assisted development: writing detailed specs, breaking work into checkable steps, and reviewing generated code to keep it aligned with the intended design.

Some of my projects to look at:

[Lightweight text-based SQL parameter binds using t-strings \(https://github.com/baverman/sqlbind-t\)](https://github.com/baverman/sqlbind-t)

[A library to help build sans-io style protocols \(https://github.com/baverman/sansproto\)](https://github.com/baverman/sansproto)

[Fast TSDB backend for Graphite \(https://github.com/baverman/hisser\)](https://github.com/baverman/hisser)

Notable experience

May 2024 — Dec 2025, Revolut, Senior Dev

An internal auth product. It covers cross-service and person authentication/authorization to grant resource access. It also includes access policy management and audit. The product includes a supporting auth library for Python/Java services, allowing developers to declare access rules for each endpoint/resource while hiding the complex details of authentication and authorization.

Stack: Java (Spark) for main auth service. Python (FastAPI) for policy management. k8s with managed PostgreSQL DB. Both services use internal ORM library.

Achievements:

- Revived Python auth library, added all features Java version had.
- Redesigned Python auth library to have a unified API across all supported web frameworks:

FastAPI/Flask/Django.

- Actively supported and drove the migration of hundreds of Python services to a new auth library.

Nov 2020 — May 2024, CloudLinux, Team Lead, remote

Lead of the KernelCare Python team (5 members). The product allows updating kernel and userspace processes without reboots and restarts.

The team maintains:

- KernelCare agent: glue logic, packaging for RHEL-based (el6, el7, el8, RedHat, CentOS, RockyLinux, AlmaLinux) and deb-based (Debian >=8, 9, 10, 11, Ubuntu >=16.04) distros.
- Scanner interface: LD_PRELOAD-based library written in C, which allows rewriting package version info to force security scanners to report proper patched versions.
- Patch server: licensing, patches access rules, patch release process.
- ePortal: on-premise solution for patch delivery for enterprise clients.
- Internal tools for tracking CVE.
- Framework to test kernel and userland patches.

Stack: python3, uWSGI, SQLite, Clickhouse, nginx, HAProxy, TincVPN, Grafana, Sentry, Docker.

Achievements:

- Designed and deployed new artifact signing scheme to be able to use periodically rotated short-term signature keys.
- Sped up agent functional tests from 1 hour to 20 minutes.
- Moved ePortal tests from real cloud VMs to Docker containers, reducing runtime from 30 minutes to 10 minutes.
- Developed and released a new forward-compatible patch metadata format for userland patches. A new patchset release doesn't depend on metadata from old releases.
- Fixed kernel test suite bugs to allow green runs.
- Implemented HA on patch server.
- Implemented HA on ePortal.
- Introduced package building in Docker containers. This allowed to build packages for non x86-64 architectures and reduced build time from 10 minutes to 30s.
- Developed and released internal SAML2 to OAuth2 proxy to allow internal services to use simple OAuth2 clients to authorize with corporate SSO.
- Refactored scanner interface with safe C practices, increased test coverage and fixed numerous segfaults. Introduced coverage measurement.
- Documented release process and introduced branch policy for ePortal and KernelCare Agent packages.

Mar 2018 — Feb 2020 (2 years), CloudLinux, Team Lead, remote

Lead of the Imunify360 backend team (3 members). The main service takes care of an event stream from all client installations (thousands of events per second, gigabytes of data per day), pushes it

through ETL pipeline to Clickhouse datastore, and handles notifications back to clients.

Stack: python3, asyncio, aiohttp, Redis, Mongo, Clickhouse, HAProxy, TincVPN, Grafana, Sentry, Docker.

Achievements:

- Reworked service architecture to be able to process a growing event stream.
- Implemented app-level real-time monitoring with in/out metrics on each step of ETL path. Statsd, Telegraf, Hisser, Grafana.
- 100% test coverage of whole ETL path, client proto parsing, queue handling, disk storage, ETL processors. As a result, team can deploy new releases directly to production without manual testing on a stage environment.
- Sped up functional tests from 30 to 10 minutes.
- Optimized ansible playbook. Deploy time shortened from 10 to 1 minute.
- Refactored a private admin site for main service (old unmaintainable code with numerous bugs).
- Implemented a fast reverse-ip lookup component.
- 20x speedup of event validation code.
- Reviewed and fixed Mongo indexes, added partial indexes to fulfill query requirements.
- Tuned HAProxy timeouts and logging, configured upstream checks.
- Moved ETL storage from Hadoop to Clickhouse and implemented Clickhouse automatic table migrations.
- Mesh VPN between production cluster nodes.
- Implemented geo-distributed RBL.

Aug 2016 — Feb 2017 (7 months), Rambler&Co, Team Lead

Lead of the Rambler/News team: 6 members, 4 backend and 2 frontend developers. There were 3 services: desktop website + mobile API, mobile site and admin control panel for the content management team. Load: 800 requests per second. My main duties were: solve performance issues and reduce technical debt.

Stack: python2, PostgreSQL, Redis, Django, flask, celery, SQLAlchemy, nginx, uWSGI.

Achievements:

- 2x codebase reduction from 40k to 20k SLOC. Code was in a messy state, endpoints simultaneously used Django/DjangoORM, Django/SQLAlchemy and Flask/SQLAlchemy with duplicated functionality. I've refactored and implemented all endpoints as Flask/SQLAlchemy views.
- Performance optimization. CPU load was reduced by 4 times. The average response latency was reduced by 3 times. I tuned DB indexes, replaced complex ORM code (hybrid props and multi-level relation joins which can lead to up to 300 db requests per HTTP request) with straightforward queries and introduced 2-level caching with uWSGI and Redis.
- Reduced deploy time from 30 to 5 minutes.
- All services were prepared for migration from FreeBSD to Linux.

May 2014 — Jun 2016 (2 years), Zvooq, Team Lead

Lead of the backend team consisting of 6 members. Zvooq is a music streaming service and our team was taking care of mobile/web API, search, editorial team tools, subscriptions, pushes, billing, third-party integrations, and backend code for custom ad campaigns.

Stack: python2, FlaskAdmin, PostgreSQL, Elasticsearch, LMDB, Redis, Werkzeug, Celery, SQLAlchemy, nginx, uWSGI, Sentry

Achievements:

- Performance optimization and major architecture rework. The main service couldn't handle 20 rps load before and can process 1000 rps after on the same hardware.
- Application real-time monitoring.
- Reduced deploy time from 20 to 1 minute. Implemented CI/CD.
- Introduced mandatory unit tests and full coverage. The team can deploy releases after automatic tests directly to production servers.

Skills

Python:

py2/py3, stdlib, asyncio, FastAPI, Flask/FlaskAdmin, SQLAlchemy, Alembic, Django, aiohttp, requests, Celery, uWSGI, lxml, cryptography, NumPy, Pandas, Cython, CFFI, ctypes, cpu/memory profilers, memory leaks.

DB:

PostgreSQL, Redis, Mongo, SQLite, LMDB, InfluxDB, Clickhouse, Elasticsearch, MySQL, Hive.

Containers:

k8s, Docker, LXC, systemd-spawn, KVM, qemu-nbd.

Tools:

Sentry, Grafana, Graphite, Jenkins, Telegraf

Linux administration:

Ansible, Fabric, systemd, nginx, HAProxy, bash scripting, pipelines, CentOS, Ubuntu, Alpine, networking, LVM.

System programming:

Process management, sockets, TCP, UDP, non-blocking IO, low-level protocol parsing.

Clouds:

GCP, AWS, Linode, OpenNebula.

VCS:

Git/StGit, Mercurial, SVN.

Frontend:

HTML, CSS, JavaScript, jQuery, React/Preact, Vue